C PROGRAMMING FROM SCRATCH · EPISODE 09

The Boolean Type

True, False, and a Bit of History

KEY POINTS

- In C, zero means false and any nonzero value means true.
- C99 added the _Bool type; stdbool.h adds the names bool, true and false.
- The name _Bool avoids clashing with programs that already defined their own bool.
- C23 makes bool, true and false keywords, but Visual Studio 2026 still needs stdbool.h.

01 Zero and nonzero

▶ VIDEO 00:49

C89 and C90 had no separate type for Boolean values. A condition counts as **false** when its value is `0`, and **true** for any other value: `1`, `2`, `100`, even `-1`.

zero-and-nonzero.c — C SOURCE

```c
#include <stdio.h>

int main()
{
    if (0)
        printf("0 is true\n");
    if (1)
        printf("1 is true\n");
    if (2)
        printf("2 is true\n");
    if (100)
        printf("100 is true\n");
    if (-1)
        printf("-1 is true\n");
    return 0;
}
```

Output

```
1 is true
2 is true
100 is true
-1 is true
```

Only the `if (0)` test prints nothing. This rule still works today, so you'll see it in a lot of existing code.

02 C99: `_Bool` and `stdbool.h`

▶ VIDEO 01:55

In **1999**, C99 added a type just for true and false: `_Bool`.

Long before C99, programmers had already created their own types called `bool`; reserving that word would have stopped their code compiling. Names that begin with an underscore and a capital letter were already reserved for the language, so `_Bool` could not clash with anyone's code.

The header `stdbool.h` adds three friendlier names; `true` and `false` are still `1` and `0` underneath. In Visual Studio 2026 it contains:

stdbool.h (excerpt) — C SOURCE

```c
#define bool  _Bool
#define false 0
#define true  1
```

03 Using `_Bool` in Visual Studio

▶ VIDEO 03:18

bool-flag.c — C SOURCE

```c
#include <stdio.h>
#include <stdbool.h>

int main()
{
    _Bool flag = true;
    if (flag)
        printf("I like C\n");
    return 0;
}
```

> WATCH OUT **Forgetting the header**
>
> Without `#include <stdbool.h>`, Visual Studio doesn't know the name `true`, and the build fails with `error C2065: 'true': undeclared identifier`.

With the header it prints `I like C`; with `false` it prints nothing, because the `printf` line is skipped.

04 Using `bool` instead

▶ VIDEO 04:37

`stdbool.h` defines `bool` as another name for `_Bool`, so `bool flag = true;` works the same way (`bool-name.c`).

A `_Bool` variable only ever stores `0` or `1`: after `bool b = 100;`, `printf("%d\n", b);` prints `1`, because any nonzero value becomes true (`bool-stores-one.c`).

05 C23: built-in keywords

▶ VIDEO 05:19

C23 makes `bool`, `true` and `false` real keywords, so a compiler with full C23 support doesn't need `stdbool.h`. GCC 15 already uses C23 by default, but Visual Studio 2026 doesn't accept these keywords yet, even with the latest C standard option (`/std:clatest`). **For this course, keep including `stdbool.h`**: it works with C99 and every later standard, including C23.

CODE 7 files on GitHub github.com/abitclearer/c-programming-from-scratch/tree/main/09-the-boolean-type